

PLSQL Exercise

Exercise 1: Control Structures

The screenshot displays the FreeSQL IDE interface. On the left, the 'Navigator' pane shows a file named 'Experiment1.sql' under the 'DATABASE' section. The main editor area contains the following SQL script:

```
1 CREATE TABLE customers (  
2   customer_id NUMBER PRIMARY KEY,  
3   name        VARCHAR2(100),  
4   age         NUMBER,  
5   balance     NUMBER,  
6   interest_rate NUMBER,  
7   is_vip      VARCHAR2(5) DEFAULT 'FALSE'  
8 );  
9  
10 CREATE TABLE loans (  
11   loan_id        NUMBER PRIMARY KEY,  
12   customer_id    NUMBER REFERENCES customers(customer_id),  
13   loan_amount    NUMBER,  
14   interest_rate  NUMBER,  
15   due_date       DATE  
16 );  
17  
18 CREATE TABLE savings_accounts (  
19
```

Below the script, the 'Script output' tab is active, showing the following output:

```
SQL> COMMIT  
  
Commit complete.  
  
Elapsed: 00:00:00.002
```

Scenario 1: The bank wants to apply a discount to loan interest rates for customers above 60 years old.

FreeSQL >_ Worksheet SQL Tutorials 23ai Connect

Navigator Files

All Files

Search files

DATABASE

- Experiment1.sql
- Scenario 1 — Loan interest discount for cus
- Scenario 2 — VIP flag for balance over \$10,0
- Scenario 3 — Loan due reminders within 30

Scenario 1 — Loan interest discount for customers over 60.sql

```
1 DECLARE
2     CURSOR cust_cursor IS
3         SELECT customer_id, age FROM customers;
4 BEGIN
5     FOR cust_rec IN cust_cursor LOOP
6         IF cust_rec.age > 60 THEN
7             UPDATE loans
8                 SET interest_rate = interest_rate - (interest_rate * 0.01)
9                 WHERE customer_id = cust_rec.customer_id;
10
11             DBMS_OUTPUT.PUT_LINE('Discount applied to customer ID: ' || cust_rec.customer_id);
12         END IF;
13     END LOOP;
14 COMMIT;
15 END;
16 /
```

Query result Script output DBMS output Explain Plan SQL history

Discount applied to customer ID: 1
Discount applied to customer ID: 3

PL/SQL procedure successfully completed.

Elapsed: 00:00:00.160

Query result Script output DBMS output Explain Plan SQL history

Discount applied to customer ID: 1
Discount applied to customer ID: 3

Scenario 2: A customer can be promoted to VIP status based on their balance

The screenshot displays the FreeSQL IDE interface. On the left, the 'Navigator' pane shows a file tree with 'Scenario 2 — VIP flag for balance over \$10,000.sql' selected. The main editor shows the following PL/SQL script:

```
1 DECLARE
2   ...CURSOR cust_cursor IS
3   ...SELECT customer_id, balance FROM customers;
4 BEGIN
5   ...FOR cust_rec IN cust_cursor LOOP
6   ...IF cust_rec.balance > 10000 THEN
7   ...UPDATE customers
8   ...SET is_vip = 'TRUE'
9   ...WHERE customer_id = cust_rec.customer_id;
10
11   ...DBMS_OUTPUT.PUT_LINE('Customer ID: ' || cust_rec.customer_id || ' marked as VIP');
12   ...END IF;
13   ...END LOOP;
14   ...COMMIT;
15 END;
```

The 'Script output' tab at the bottom shows the following output:

```
Customer ID 1 marked as VIP
Customer ID 2 marked as VIP

PL/SQL procedure successfully completed.
Elapsed: 00:00:00.015
```

This close-up view of the 'DBMS output' tab shows the following output:

```
Customer ID 1 marked as VIP
Customer ID 2 marked as VIP
```

Scenario 3: The bank wants to send reminders to customers whose loans are due within the next 30 days.

The screenshot shows the FreeSQL IDE interface. On the left, the 'Navigator' pane displays a file tree under 'DATABASE' with files: 'Experiment1.sql', 'Scenario 1 — Loan interest discount for cus', 'Scenario 2 — VIP flag for balance over \$10,', and 'Scenario 3 — Loan due reminders within 30'. The main editor shows a PL/SQL script titled 'Scenario 3 — Loan due reminders within 30 days.sql'. The script declares a cursor, selects loan details, and loops through them to generate reminders for loans due within 30 days. The output pane at the bottom shows the script was executed successfully, displaying a reminder for Customer Alice and the elapsed time of 00:00:00.160.

```
1 DECLARE
2   ...CURSOR loan_cursor IS
3   ...SELECT l.loan_id, l.customer_id, l.due_date, c.name
4   ...FROM loans l
5   ...JOIN customers c ON l.customer_id = c.customer_id
6   ...WHERE l.due_date BETWEEN SYSDATE AND SYSDATE + 30;
7 BEGIN
8   ...FOR loan_rec IN loan_cursor LOOP
9   ...   DBMS_OUTPUT.PUT_LINE('Reminder: Customer: ' || loan_rec.name ||
10   ...   'has a loan (ID: ' || loan_rec.loan_id ||
11   ...   ') due on: ' || TO_CHAR(loan_rec.due_date, 'DD-MON-Y
12   ...END LOOP;
13 END;
```

Query result **Script output** DBMS output Explain Plan SQL history

Reminder: Customer Alice has a loan (ID 101) due on 08-JUL-2026

PL/SQL procedure successfully completed.

Elapsed: 00:00:00.160

This is a close-up of the 'DBMS output' pane. It shows the reminder message: 'Reminder: Customer Alice has a loan (ID 101) due on 08-JUL-2026'. There are icons for clearing, downloading, and copying the output.

Query result Script output **DBMS output** Explain Plan SQL history

Reminder: Customer Alice has a loan (ID 101) due on 08-JUL-2026